

Product Requirements & Business Requirements Document

Project Name

[Working Product Name] – Free-First Anonymous Dating & Connection Platform

Document Type: Business Requirements Document (BRD) + Product Requirements Document (PRD) + Technical Implementation Plan

Initial Architecture: Modular Monolith

Backend: Java + Spring Boot

Frontend: React.js + Tailwind CSS

Database: MySQL

Initial Business Model: Free for all users

Future Business Model: Freemium / Subscription / Premium Features

Primary Platform: Web Application

Future Platforms: Android / iOS / PWA

1. Executive Summary

The proposed application is a modern dating and social-connection platform designed around a simple principle:

People should be able to connect and have a conversation before being pressured to reveal their complete dating profile.

Traditional dating applications generally follow a profile-first model:

1. Create profile.
2. Upload photographs.
3. Browse other profiles.
4. Like/dislike.
5. Match.

6. Chat.

This product introduces an alternative flow:

1. Create a basic verified account.
2. Enter an anonymous discovery/chat environment.
3. Connect with another compatible person.
4. Start a conversation without immediately revealing the complete profile.
5. Continue talking if both people are comfortable.
6. Either person may request profile revelation.
7. The other person can accept or decline.
8. If both mutually agree, selected profile information is revealed.
9. Users can then continue the relationship through normal profile-based interaction.

The objective is not simply to create another Tinder-like application.

The objective is to create a platform focused on:

- conversation before appearance,
- mutual consent,
- privacy,
- safety,
- genuine connections,
- low-pressure interaction,
- free accessibility,
- and eventually a sustainable premium business model.

The initial version will be completely free to maximize adoption, network effects, user growth and brand awareness.

Once the platform reaches sufficient scale and user engagement, premium functionality can be introduced without destroying the core free experience.

2. Product Vision

Vision Statement

To build a trusted social and dating platform where people can discover meaningful connections through conversation, privacy and mutual consent rather than relying exclusively on photographs and superficial profile browsing.

Mission

The platform aims to make online dating:

- more accessible,
 - less intimidating,
 - more private,
 - more conversation-oriented,
 - safer,
 - and more meaningful.
-

3. Core Product Philosophy

The product should be built around five principles.

3.1 Conversation First

The application should encourage users to know another person before making decisions based entirely on appearance.

3.2 Consent First

Profile revelation should be mutual.

One user should never be able to force another user to reveal information.

3.3 Privacy First

Users should have control over:

- what information is visible,
- when information is visible,
- who can see it,
- and whether they can be discovered.

3.4 Safety First

Anonymous communication creates additional safety challenges.

Therefore, anonymity should never mean:

"Users can behave without consequences."

The system should maintain internal account identity, reporting mechanisms, moderation systems, abuse detection and appropriate audit information while keeping that information hidden from ordinary users.

3.5 Free-First Growth

The core platform should initially be free.

The objective is to create:

Users → Conversations → Connections → Network Effect → Brand → Monetization

rather than:

Users → Paywall → Low adoption

4. Problem Statement

Existing dating applications often have several problems:

Profile-first interaction

Users frequently make decisions based heavily on:

- photographs,
- appearance,
- age,
- occupation,
- location,
- short biographies.

Conversation pressure

Users may feel pressure to match and immediately maintain a conversation.

Privacy concerns

Users may not want their identity, photographs or personal information immediately visible to strangers.

Fake profiles

Dating platforms are vulnerable to:

- fake accounts,
- bots,
- scams,

- impersonation,
- spam.

Superficial discovery

A profile-based system can encourage users to rapidly swipe through large numbers of profiles instead of developing meaningful conversations.

Cost barrier

Many dating platforms place useful features behind subscriptions.

The proposed product attempts to address these problems through anonymous, consent-driven conversation.

5. Target Users

Primary User Group

Adults aged 18+ looking for:

- romantic relationships,
- meaningful conversations,
- friendships,
- social connections,
- long-term relationships.

Secondary User Group

Users who:

- are uncomfortable with traditional dating applications,
- prefer conversation before sharing photographs,
- value privacy,
- want a less appearance-driven experience.

Future User Segmentation

The platform can eventually support different connection intentions:

- Serious relationship
- Casual dating
- Friendship
- Conversation

- Networking/social connection

However, the product should clearly distinguish dating from general social interaction to reduce misuse.

6. Age Requirement

The initial product should be strictly **18+**.

The system should require the user to acknowledge that they are an adult.

Age-related controls should exist at:

- registration,
- profile creation,
- moderation,
- reporting,
- account enforcement.

The product should not attempt to support minors in the initial platform.

7. Core User Journey

7.1 Registration

User opens the application.

They select:

Create Account

Possible registration methods:

- Email
- Mobile number
- Future: Google/Apple/social login

Initially, OTP infrastructure may not be available.

Therefore, development should support a pluggable verification architecture.

Example:

```
VerificationService
|
+-- EmailOtpProvider
|
+-- SmsOtpProvider
|
+-- DevelopmentOtpProvider
```

During development, a development OTP provider can be used.

Production OTP providers can later be integrated without changing the registration business logic.

8. Account Creation

User provides:

- First name / display name
- Date of birth
- Gender
- Interested in
- Email
- Mobile number
- Password
- Country
- Optional location
- Terms acceptance
- Privacy policy acceptance
- Community guidelines acceptance

The system creates:

```
User Account
  ↓
Verification
  ↓
Basic Profile
  ↓
Onboarding
  ↓
Discovery
```

9. Profile System

The profile system should be divided into different visibility levels.

9.1 Private Information

Never publicly exposed.

Examples:

- Password
- Internal user ID
- OTP information
- Login metadata
- Security information
- Moderation information

9.2 Basic Profile

Potentially visible after mutual revelation.

Examples:

- Display name
- Age
- General location
- Bio
- Interests
- Dating intention

9.3 Sensitive Profile

Requires explicit user control.

Examples:

- Exact location
- Social media accounts
- Phone number
- Email
- Workplace
- Other personal information

The system should discourage users from publishing sensitive contact information publicly.

10. Anonymous Chat Concept

This is the primary product differentiator.

Anonymous Chat Mode

A user can enter an anonymous discovery environment.

Instead of immediately seeing a complete profile, the system presents a conversation opportunity.

Example:

You have been connected with someone.

Conversation Topic:

"What is one thing you could talk about for hours?"

Start Conversation

The other person can receive:

Someone wants to start an anonymous conversation with you.

Accept

Decline

If accepted:

Anonymous User

|

|

Chat

|

|

Anonymous User

Neither user initially sees the complete dating profile.

11. Anonymous Identity

Anonymous does not mean completely untraceable.

Internally:

```
Anonymous User
  ↓
Internal User ID
  ↓
Moderation / Security System
```

Externally:

```
Anonymous User A
Anonymous User B
```

The platform should not expose internal identifiers.

The anonymous identity may have a generated nickname such as:

- BlueFox
- QuietMoon
- CuriousPanda
- CoffeeSoul

However, the names should not become a substitute for actual identity verification.

12. Conversation Matching

The initial matching algorithm can be relatively simple.

Matching Factors

Potential factors:

- Age range
- Gender preference
- General location
- Dating intention
- Interests

- Language
- Activity level
- Conversation topics
- User preferences
- Previous interactions
- Block/report history

A first version does not require sophisticated AI.

MVP algorithm

```
Candidate Users
  ↓
Eligibility Filter
  ↓
Preference Filter
  ↓
Safety Filter
  ↓
Availability Filter
  ↓
Interest Compatibility
  ↓
Randomized Ranking
  ↓
Conversation Candidate
```

Randomization is important because otherwise highly active users may dominate discovery.

13. Mutual Profile Revelation

This is one of the most important product features.

At any point during an anonymous conversation:

User A:

Reveal my profile

The system sends:

```
User A wants to reveal profiles.
```

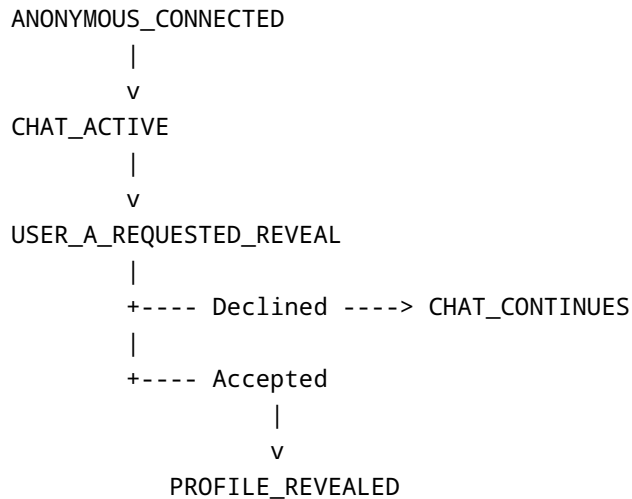
Do you want to reveal your profile too?

[Reveal Profile]

[Keep Anonymous]

Only if User B accepts should the profile transition occur.

State Machine



If both users independently request revelation:

User A Request
+
User B Request
=
Profile Revealed

14. Profile Revelation Levels

Instead of a binary reveal, the platform can eventually support levels.

Level 0 – Anonymous

No profile.

Level 1 – Basic

Show:

- First name/display name
- Age
- General location
- Interests

Level 2 – Social Profile

Show:

- Profile photo
- Bio
- More interests
- Dating intentions

Level 3 – Direct Connection

Users can decide to exchange additional contact information.

This must always require explicit consent.

15. Chat System

Chat should support:

MVP

- Text messages
- Message timestamps
- Delivered status
- Read status
- Block
- Report
- Delete conversation
- End conversation

Future

- Images
- Voice messages
- Reactions
- GIFs

- Stickers
- Voice/video calls
- Shared interests
- Conversation games

Multimedia messaging should not be included until moderation and storage systems are sufficiently mature.

16. Conversation Ending

Either user should be able to select:

End Conversation

Options:

- End conversation
- Block user
- Report user

The other user should receive a neutral message such as:

This conversation has ended.

Do not reveal unnecessary information about the reason unless the user voluntarily chooses to provide it.

17. Traditional Dating Mode

The platform can also offer a conventional profile discovery mode.

Example:

Discovery

[Profile Card]

Age

Location

Interests

Bio

Like
Pass

This gives users two different experiences.

Mode A

Anonymous Conversation

Conversation first.

Mode B

Profile Discovery

Profile first.

Users can choose whichever experience they prefer.

18. Matching System

Traditional matching can work as:

```
User A likes User B
+
User B likes User A
=
MATCH
```

After a match:

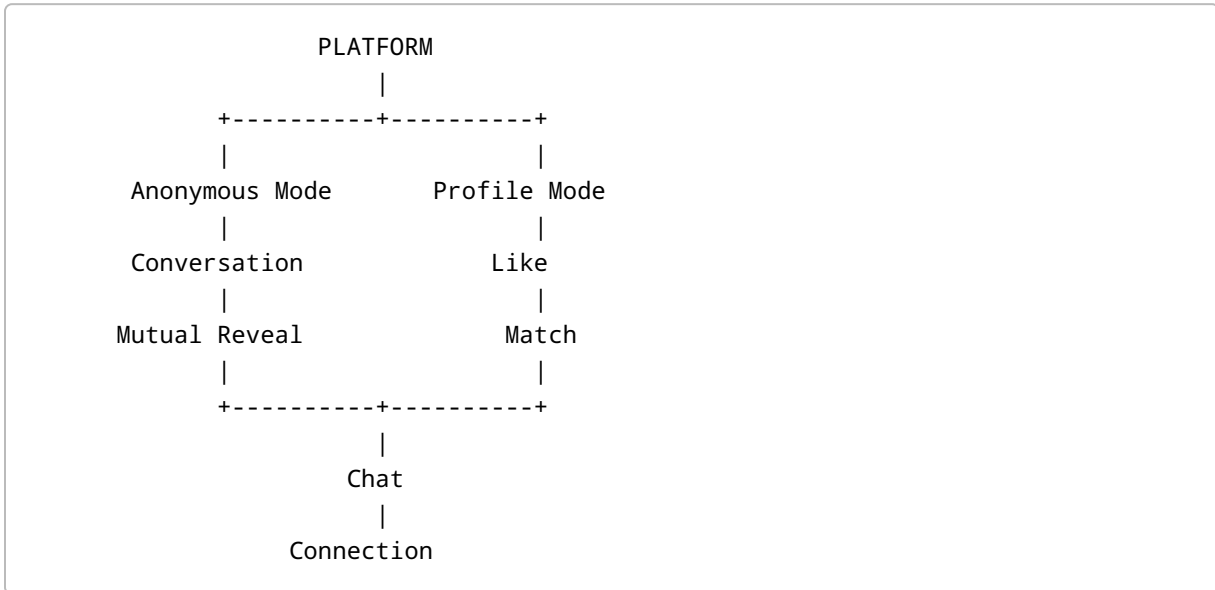
```
MATCH
  ↓
CHAT
  ↓
CONTINUE
```

The anonymous system can remain separate.

19. Hybrid Connection Model

Eventually the platform can combine both systems.

Example:



This is a strong product proposition because users are not forced into one dating style.

20. Home Screen

The home screen should be simple.

Example:

Welcome, Ganesh

What would you like to do?

Anonymous Conversations

Meet someone through conversation first.

[Start Conversation]

Discover Profiles
Explore people nearby.

[Discover]

Messages
Matches
Profile
Settings

21. Dashboard

The authenticated dashboard can contain:

- Anonymous conversations
- Matches
- Messages
- Profile completion
- Verification status
- Notifications
- Safety center
- Settings

22. User Profile

Profile page:

Profile

Display Name
Age
Location

About Me

Interests

Looking For

Profile Photos

Verification Status

Profile Visibility

[Edit Profile]

23. Profile Privacy Controls

Users should control:

Discoverability

- Everyone
- Preferences only
- Nobody

Photo visibility

- Public
- After mutual reveal

Location

Never display exact GPS coordinates.

Display only approximate information such as:

- City
- District
- Region

Online status

Optional.

Last active

Optional.

24. Notification System

Notifications can include:

- New anonymous conversation request
- New message
- Profile reveal request
- Profile reveal accepted
- New match
- Safety notification
- Account verification
- Login/security alert

Notification architecture should support:

```
NotificationService
|
+-- In-App
+-- Email
+-- SMS
+-- Push Notification
```

25. Verification System

Verification should be designed as a pluggable subsystem.

Email Verification

```
User
↓
Request OTP
↓
Email OTP Provider
↓
OTP
↓
Verification
```

Mobile Verification

```
User
↓
Request OTP
↓
SMS Provider
↓
OTP
↓
Verification
```

Development Mode

Since operational APIs may not yet be available:

```
DevelopmentOtpProvider
```

can return or log a development OTP.

Example:

```
OTP: 123456
```

This should be enabled only under development configuration.

Never ship hard-coded development OTP behavior into production.

26. OTP Architecture

Create an abstraction:

```
public interface OtpService {

    void sendOtp(String destination);

    boolean verifyOtp(String destination, String otp);
}
```

Production implementation:

```
SmsOtpService  
EmailOtpService
```

Development implementation:

```
MockOtpService
```

This allows providers to be changed later without changing registration logic.

27. Security Requirements

Security should be considered from the first version rather than added later.

Authentication

Use:

- Spring Security
- Secure password hashing
- JWT or secure session-based authentication
- Refresh-token strategy if JWT is selected
- Token expiration
- Account logout/revocation

Password Security

Passwords must never be stored as plaintext.

Use a strong password hashing algorithm such as:

```
Argon2id
```

or an appropriately configured BCrypt implementation.

28. Account Protection

Implement:

- Login rate limiting
 - OTP rate limiting
 - Password reset
 - Account lockout/risk controls
 - Suspicious login detection
 - Session management
 - Device/session management
 - CSRF protection where applicable
 - Secure HTTP headers
 - Input validation
 - Output encoding
 - SQL injection protection
 - XSS protection
 - Authorization checks
-

29. Anonymous Chat Security

This is particularly important.

Anonymous users must not be able to discover:

- internal user IDs,
- email addresses,
- phone numbers,
- IP addresses,
- exact locations,
- authentication information,
- database IDs.

Never expose database IDs directly if avoidable.

Use public identifiers such as UUIDs.

Example:

```
Internal:  
user_id = 183729
```

```
Public:  
user_public_id = UUID
```

30. Abuse Prevention

The anonymous system creates potential abuse risks.

Therefore implement:

Block

A user can immediately block another user.

Report

Report categories:

- Harassment
- Spam
- Scam
- Sexual harassment
- Threatening behavior
- Fake profile
- Impersonation
- Inappropriate content
- Other

Automatic restrictions

Repeated reports can trigger:

```
Warning  
  ↓  
Temporary restriction  
  ↓  
Manual review  
  ↓  
Suspension  
  ↓  
Permanent ban
```

Do not automatically permanently ban users based solely on a single unverified report.

31. Moderation System

Create an administrative moderation system.

Admins should be able to see:

- User account
- Reports
- Report history
- Conversation metadata
- Account status
- Verification status
- Moderation actions

Access to sensitive chat content should be tightly controlled and audited.

32. Admin Panel

Admin dashboard:

Admin Dashboard

Users

Reports

Moderation

Verification

Blocked Accounts

Analytics

System Health

Feature Flags

Configuration

Subscriptions

Payments

33. Admin Roles

Possible roles:

SUPER_ADMIN

Complete access.

ADMIN

General platform administration.

MODERATOR

User/report moderation.

SUPPORT

Customer support without unrestricted administrative privileges.

Use Role-Based Access Control.

34. Database Design

Initial database:

MySQL

The database should be designed around business domains rather than one giant set of tables.

Possible tables:

```
users
user_credentials
user_verifications
user_profiles
user_preferences
user_interests
interests
user_photos
user_privacy_settings
user_sessions
otp_requests

anonymous_sessions
anonymous_connections
anonymous_reveal_requests

conversations
conversation_participants
messages
```

```
likes
matches

blocks
reports
moderation_actions

notifications
notification_preferences

subscriptions
subscription_plans
payments

audit_logs
```

35. Important Database Principle

Avoid coupling every module to every other table.

For example:

```
Anonymous Chat
  |
  v
anonymous_connections
  |
  v
conversation
```

Rather than allowing anonymous chat logic to directly depend on every profile table.

This makes future microservice extraction much easier.

36. Suggested Core Entities

User

```
User
- id
```

- publicId
- email
- mobile
- passwordHash
- status
- emailVerified
- mobileVerified
- createdAt
- updatedAt

UserProfile

```
UserProfile
- id
- userId
- displayName
- dateOfBirth
- gender
- bio
- city
- relationshipIntent
- profileVisibility
```

UserPreference

```
UserPreference
- id
- userId
- preferredGender
- minAge
- maxAge
- preferredDistance
```

Conversation

```
Conversation
- id
- publicId
- type
- status
- createdAt
- endedAt
```

Message

```
Message
- id
- conversationId
- senderId
- content
- messageType
- createdAt
- deletedAt
```

37. Anonymous Connection Entity

```
AnonymousConnection
- id
- conversationId
- userAId
- userBId
- status
- revealStatus
- createdAt
- endedAt
```

Possible statuses:

```
PENDING
ACTIVE
REVEAL_REQUESTED
REVEALED
ENDED
BLOCKED
```

38. API Architecture

The backend should follow:

```
Controller
↓
```

Application Service



Domain Service



Repository



Database

Avoid putting business logic directly into controllers.

Bad:

Controller



Repository

Preferred:

Controller



Service



Repository

39. REST API Structure

Example:

```
/api/v1/auth  
/api/v1/users  
/api/v1/profiles  
/api/v1/preferences  
/api/v1/discovery  
/api/v1/anonymous  
/api/v1/conversations  
/api/v1/messages  
/api/v1/matches  
/api/v1/notifications  
/api/v1/reports
```

```
/api/v1/blocks  
/api/v1/settings
```

Admin:

```
/api/v1/admin/users  
/api/v1/admin/reports  
/api/v1/admin/moderation  
/api/v1/admin/analytics
```

40. Example Authentication APIs

```
POST /api/v1/auth/register  
  
POST /api/v1/auth/login  
  
POST /api/v1/auth/logout  
  
POST /api/v1/auth/refresh  
  
POST /api/v1/auth/email/request-otp  
  
POST /api/v1/auth/email/verify-otp  
  
POST /api/v1/auth/mobile/request-otp  
  
POST /api/v1/auth/mobile/verify-otp  
  
POST /api/v1/auth/forgot-password  
  
POST /api/v1/auth/reset-password
```

41. Anonymous Chat APIs

```
POST /api/v1/anonymous/discover  
  
POST /api/v1/anonymous/request  
  
POST /api/v1/anonymous/request/{id}/accept
```

```
POST /api/v1/anonymous/request/{id}/decline

POST /api/v1/anonymous/{connectionId}/reveal/request

POST /api/v1/anonymous/{connectionId}/reveal/accept

POST /api/v1/anonymous/{connectionId}/reveal/decline

POST /api/v1/anonymous/{connectionId}/end
```

42. Messaging APIs

For the initial MVP, REST can handle:

```
GET /api/v1/conversations

GET /api/v1/conversations/{id}/messages

POST /api/v1/conversations/{id}/messages

DELETE /api/v1/messages/{id}
```

For real-time communication, use:

```
WebSocket
```

with Spring WebSocket/STOMP or another appropriate real-time architecture.

43. Real-Time Chat Architecture

Initial architecture:

```
React
|
WebSocket
|
Spring Boot
|
```

```
Chat Service
|
MySQL
```

At larger scale:

```
React
|
Load Balancer
|
WebSocket Gateway
|
Redis Pub/Sub
|
Chat Service
|
Message Storage
```

Eventually message storage may be separated from transactional relational data.

44. Frontend Architecture

Recommended:

```
React.js
TypeScript
Tailwind CSS
React Router
TanStack Query
Zustand or Redux Toolkit
Axios/fetch
WebSocket client
```

TypeScript is strongly recommended over plain JavaScript for a product of this complexity.

45. Frontend Structure

Example:

```
src/
|
├─ app/
├─ components/
├─ layouts/
├─ pages/
|
├─ features/
|   ├─ auth/
|   ├─ profile/
|   ├─ discovery/
|   ├─ anonymous-chat/
|   ├─ messaging/
|   ├─ matches/
|   ├─ notifications/
|   └─ settings/
|
├─ services/
├─ hooks/
├─ store/
├─ types/
├─ utils/
└─ constants/
```

This mirrors the backend domain structure.

46. Backend Modular Monolith

The initial Spring Boot application should NOT become one huge package.

Instead:

```
com.application
|
├─ auth
├─ user
├─ profile
├─ discovery
├─ matching
├─ anonymouschat
├─ conversation
└─ messaging
```

```
|— notification
|— moderation
|— subscription
|— payment
|— common
```

Each module should have clear responsibilities.

47. Why Modular Monolith First?

A microservices architecture from day one introduces:

- operational complexity,
- service discovery,
- network failures,
- distributed transactions,
- deployment complexity,
- monitoring requirements,
- increased infrastructure costs.

For an early-stage product, these problems are usually unnecessary.

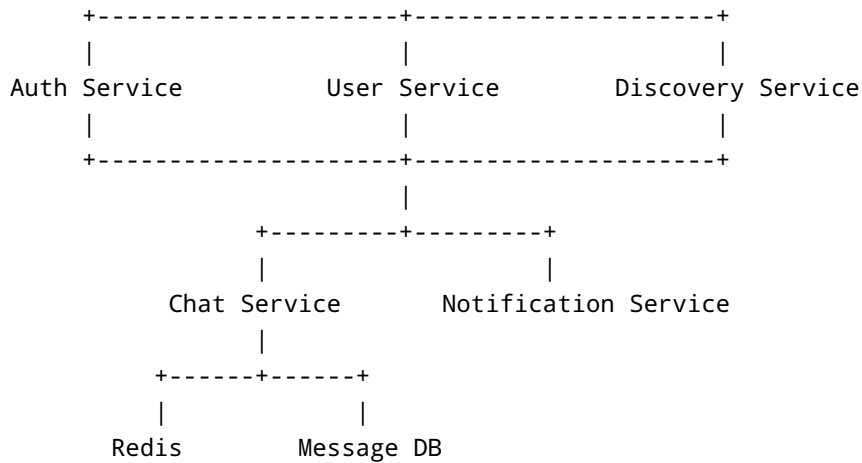
A modular monolith provides:

```
Simple deployment
+
Simple development
+
Fast iteration
+
Clear module boundaries
+
Future scalability
```

48. Future Microservice Architecture

Once the platform grows:

```
API Gateway
|
```



Potential future services:

```

Authentication Service
User Service
Profile Service
Matching Service
Discovery Service
Anonymous Chat Service
Messaging Service
Notification Service
Moderation Service
Media Service
Subscription Service
Payment Service
Analytics Service
  
```

Do not extract all of these immediately.

Extract services only when scale or organizational needs justify them.

49. Microservice Extraction Strategy

The modular monolith should be designed so modules communicate through interfaces/events.

For example:

```

ProfileModule
  |
ProfileUpdatedEvent
  
```

```

      |
Event Bus
      |
DiscoveryModule

```

Initially the event can be handled internally.

Later:

```
Kafka
```

or another message broker can transport it between services.

50. Event-Driven Future Architecture

Potential events:

```
UserRegistered
UserVerified
ProfileCreated
ProfileUpdated
ConversationCreated
MessageSent
RevealRequested
ProfileRevealed
MatchCreated
UserReported
UserBlocked
UserSuspended
SubscriptionStarted
PaymentCompleted

```

This makes the application easier to scale.

51. Caching

Initially:

MySQL

may be sufficient.

Later:

Redis

can be introduced for:

- sessions,
- OTP rate limits,
- temporary matching queues,
- online presence,
- frequently accessed profiles,
- feature flags,
- rate limiting,
- WebSocket coordination.

52. Search

Initially use MySQL queries.

At larger scale, introduce a search engine if required.

Possible future architecture:

MySQL

|

+---- Source of Truth

Search Index

|

+---- Discovery/Search

Do not introduce Elasticsearch/OpenSearch unless actual requirements justify it.

53. Image Storage

Do not store large profile images directly in MySQL.

Use object storage:

```
React
↓
Upload API
↓
Object Storage
↓
Image URL
↓
MySQL
```

The database stores metadata rather than image binary data.

Future object-storage providers can be selected according to cost, geography and scale.

54. Image Processing

Eventually implement:

- resizing,
 - compression,
 - thumbnails,
 - metadata stripping,
 - content moderation,
 - file type validation,
 - maximum file size,
 - malicious file protection.
-

55. Location Architecture

Location should be privacy-friendly.

Do not store or expose precise location unnecessarily.

Possible levels:

```
Country
State
City
Approximate Area
```

For matching:

```
User coordinates
      ↓
Distance calculation
      ↓
Approximate matching
```

Exact coordinates should not be exposed to another user.

56. Recommendation Engine

The first recommendation engine can use weighted scoring.

Example:

```
Compatibility Score =

Interest Match      25%
Age Preference      20%
Relationship Intent  20%
Location            15%
Activity             10%
Conversation History 10%
```

The exact weights should be configurable.

Later, data can be used to improve recommendations.

57. Avoid Algorithmic Bias

Do not optimize only for:

Messages
Likes
Time spent

because those metrics can encourage unhealthy engagement.

Also measure:

- meaningful conversations,
- mutual connections,
- successful profile revelations,
- user satisfaction,
- report rates,
- blocking rates,
- retention,
- healthy interaction patterns.

58. Safety-by-Design

The platform should include safety features from MVP.

Required:

- Block
- Report
- Unmatch/end conversation
- Account suspension
- Community guidelines
- Privacy controls
- Abuse monitoring
- Rate limiting
- Moderation dashboard
- Security logs

Future:

- AI-assisted moderation
 - Scam detection
 - Bot detection
 - Image moderation
 - Suspicious behavior detection
 - Identity verification
-

59. Anti-Bot System

Anonymous chat can be attractive to bots.

Potential defenses:

```
Rate Limiting
+
CAPTCHA where appropriate
+
Email/mobile verification
+
Behavior analysis
+
Message frequency limits
+
Account age signals
```

Do not make CAPTCHA mandatory for every action if it creates excessive user friction.

Use risk-based controls.

60. Anti-Scam Features

Dating platforms can be targeted by financial scams.

Potential detection signals:

- repetitive messages,
- suspicious links,
- payment requests,
- crypto investment language,
- requests for money,
- external messaging migration immediately after connection,
- mass messaging,
- repeated account creation.

The system can warn users:

Never send money or financial information to someone you met through this platform.

61. Community Guidelines

The platform should have clearly visible rules.

Examples:

Prohibited

- Harassment
 - Threats
 - Hate-based abuse
 - Sexual harassment
 - Non-consensual content
 - Scams
 - Impersonation
 - Spam
 - Financial fraud
 - Illegal activity
 - Doxxing
 - Sharing private information without permission
-

62. Reporting Workflow

```
User Reports Account
↓
Report Created
↓
Risk Classification
↓
Automated Checks
↓
Moderator Queue
↓
Action
```

Actions:

```
No Action
Warning
Content Removal
Temporary Restriction
```

Temporary Suspension
Permanent Ban

63. Business Model

Phase 1 – Completely Free

Goal:

Growth

Potential revenue:

None or minimal

The priority is:

- user acquisition,
- retention,
- engagement,
- brand awareness,
- network effects.

64. Phase 2 – Freemium

Once the platform has a meaningful user base, introduce premium functionality.

The fundamental ability to meet people should remain available for free.

Premium features might include:

- Advanced discovery filters
- Additional daily discovery
- Advanced matching preferences
- Profile boosts
- Read receipts
- Incognito discovery
- Advanced profile controls
- Priority discovery
- Conversation tools

- Profile analytics

The exact premium feature set should be validated through user research rather than assuming users will pay.

65. Phase 3 – Subscription

Possible plans:

Free
Premium
Premium+

Example:

Free

- Profile
- Basic discovery
- Anonymous conversations
- Basic messaging
- Basic matching

Premium

- Advanced filters
- Additional discovery
- Profile boost
- Incognito mode
- Advanced preferences

Premium+

- Premium discovery
- Priority visibility
- Advanced recommendations
- Additional privacy features

Pricing should be determined after market validation.

66. Monetization Principle

Do NOT make safety or fundamental communication unnecessarily paywalled.

Avoid:

Pay to send a basic message.

Instead monetize:

convenience, discovery, visibility and advanced controls.

This preserves the original free-platform identity.

67. Growth Strategy

The platform has a potential viral loop.

```
graph TD; A[User joins] --> B[Starts conversation]; B --> C[Makes connection]; C --> D[Invites friends]; D --> E[More users]; E --> F[Better matching pool]; F --> G[More successful connections]; G --> H[More invitations];
```

User joins
↓
Starts conversation
↓
Makes connection
↓
Invites friends
↓
More users
↓
Better matching pool
↓
More successful connections
↓
More invitations

68. Initial Geographic Strategy

Do not launch globally without sufficient density.

A dating platform has a network-effect problem:

A user needs enough relevant people nearby.

Therefore, the launch should ideally focus on a limited geographic market first.

Once there is sufficient user density:

```
graph TD;
    A[Market A] --> B[Market B];
    B --> C[Market C];
    C --> D[National];
    D --> E[International];
```

69. Product Growth Metrics

Track:

Acquisition

- Registrations
- Registration conversion
- Verification completion

Activation

- Profile completion
- First discovery
- First anonymous conversation
- First message

Engagement

- Messages per active user
- Conversations per user
- Active days

Connection

- Mutual conversations
- Profile revelation rate
- Matches
- Continued conversations

Retention

- Day 1

- Day 7
- Day 30

Safety

- Reports per 1,000 users
 - Blocks
 - Spam detection
 - Abuse rate
 - Fake account rate
-

70. North Star Metric

A strong candidate for the initial North Star Metric is:

Meaningful Mutual Connections per Active User

rather than simply:

Number of messages sent.

A meaningful connection could be defined as a conversation where:

- both users actively participate,
- conversation continues beyond an initial threshold,
- neither user immediately blocks/reports the other,
- and optionally both agree to reveal their profiles.

The definition should evolve based on product research.

71. MVP Scope

The MVP should NOT attempt to build everything.

MVP Features

Authentication

- Registration
- Login
- Logout
- Password reset
- Development OTP

- Email/mobile verification architecture

Profile

- Create profile
- Edit profile
- Basic preferences
- Profile photo
- Privacy settings

Discovery

- Basic profile discovery
- Anonymous discovery

Anonymous Chat

- Conversation request
- Accept
- Decline
- Messaging
- Profile reveal request
- Mutual profile reveal
- End conversation
- Block
- Report

Traditional Matching

- Like
- Pass
- Mutual match
- Chat

Safety

- Report
- Block
- Account status

Admin

- Login
- User management
- Reports
- Moderation
- Account suspension

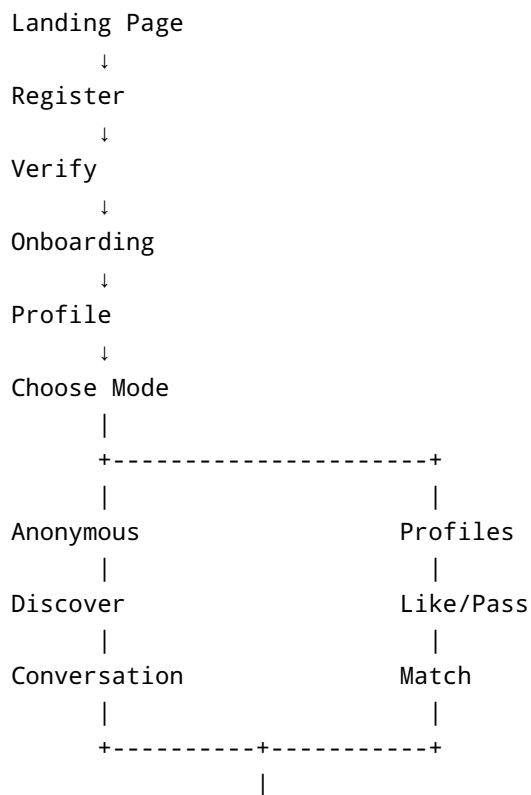
72. Features NOT Required for MVP

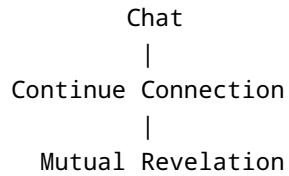
Do not initially build:

- Video calls
- Voice calls
- AI dating coach
- Complex recommendation AI
- Stories
- Live streaming
- Gifts
- Cryptocurrency
- Marketplace
- Advanced social feed
- Complex subscription system
- Multiple microservices
- Large-scale analytics infrastructure

These can come later.

73. MVP User Flow





74. Landing Page

The landing page should communicate the differentiator immediately.

Possible headline:

Meet through conversation. Reveal when you're ready.

Supporting message:

A free dating platform where you can talk first, connect naturally, and choose when to reveal your profile.

Primary CTA:

Start Connecting

Secondary CTA:

How It Works

75. Product UX Philosophy

The UI should feel:

- modern,
- clean,
- welcoming,
- trustworthy,
- private,
- lightweight.

Avoid copying Tinder/Bumble visual identity too closely.

The product should develop its own brand.

76. Frontend Design System

Recommended initial technology:

```
React
+
TypeScript
+
Tailwind CSS
```

Use reusable components:

```
Button
Input
Modal
Dialog
Avatar
ProfileCard
ChatBubble
Toast
Dropdown
Tabs
BottomNavigation
LoadingState
EmptyState
ErrorState
```

77. Responsive Design

The website must work well on:

- desktop,
- laptop,
- tablet,
- mobile browser.

Mobile-first CSS should be preferred.

The architecture should eventually allow conversion into:

```
PWA
Android
iOS
```

without redesigning the backend APIs.

78. Backend Technology Stack

Recommended:

```
Java 21+
Spring Boot
Spring Security
Spring Data JPA
Hibernate
MySQL
Flyway
Bean Validation
WebSocket
OpenAPI / Swagger
JUnit
Mockito
Testcontainers
Maven or Gradle
```

The exact Java/Spring versions should be pinned to supported stable releases when implementation begins.

79. Infrastructure

Initial environment:

```
Frontend
|
Backend
|
MySQL
```

Separate environments:

```
Development
Testing
Staging
Production
```

Never develop directly against production.

80. Configuration

Use environment-based configuration.

Example:

```
application-dev.yml
application-test.yml
application-staging.yml
application-prod.yml
```

Secrets should NOT be committed into Git.

Use:

```
Environment Variables
Secret Manager
```

for production credentials.

81. Database Migration

Use:

```
Flyway
```

or an equivalent database migration system.

Never rely on manually changing production database schemas.

Example:

```
V1__create_users.sql
V2__create_profiles.sql
V3__create_conversations.sql
V4__create_messages.sql
```

82. API Documentation

Use OpenAPI/Swagger.

Every public API should define:

- request,
- response,
- authentication requirement,
- error responses,
- validation rules.

Example:

```
POST /api/v1/anonymous/{id}/reveal/request

200 OK
400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found
409 Conflict
429 Too Many Requests
```

83. Error Handling

Use consistent API responses.

Example:

```
{
  "success": false,
  "error": {
    "code": "USER_NOT_FOUND",
```

```
    "message": "The requested user could not be found."
  },
  "timestamp": "...
}
```

Do not expose internal stack traces to users.

84. Logging

Logs should contain:

- request ID,
- timestamp,
- endpoint,
- response status,
- execution time,
- relevant technical information.

Avoid logging:

- passwords,
 - OTP values,
 - tokens,
 - private messages unnecessarily,
 - sensitive personal information.
-

85. Audit Logging

Certain actions should be auditable:

```
Login
Logout
Password reset
Email verification
Mobile verification
Profile changes
Report creation
Block
Admin suspension
Admin deletion
Profile reveal
```

Subscription purchase
Payment event

Audit logs should be protected from ordinary users.

86. Testing Strategy

Testing should exist at multiple levels.

Unit Tests

Test:

- services,
- validators,
- matching logic,
- reveal logic,
- authorization.

Integration Tests

Test:

- database,
- repositories,
- REST APIs,
- authentication.

End-to-End Tests

Test:

Registration
→ Login
→ Profile
→ Discovery
→ Anonymous Connection
→ Chat
→ Reveal

87. Critical Business Rules to Test

Reveal Rule

```
One user requests reveal  
≠  
Profile automatically revealed
```

Only mutual consent reveals profiles.

Block Rule

```
Blocked User  
≠  
Discoverable
```

Report Rule

Reporting must not automatically expose the reporter's identity.

Age Rule

Only eligible adult accounts can use dating functionality.

Authentication Rule

Unverified accounts should have restricted functionality according to configured verification policy.

88. CI/CD

Recommended pipeline:

```
Git Push  
↓  
Build  
↓  
Unit Tests  
↓  
Integration Tests  
↓  
Security Checks
```

```
↓  
Build Docker Image  
↓  
Deploy Staging  
↓  
Approval  
↓  
Production
```

89. Docker

The application should be container-ready.

Possible structure:

```
frontend  
backend  
mysql  
redis
```

Development can use Docker Compose.

Production infrastructure can evolve later.

90. Scalability Strategy

Stage 1

```
React  
Spring Boot Monolith  
MySQL
```

Stage 2

```
React  
Spring Boot Modular Monolith  
MySQL
```

Redis
Object Storage

Stage 3

Load Balancer
Spring Boot Application Instances
MySQL
Redis
Object Storage
Queue

Stage 4

API Gateway
Auth Service
User Service
Discovery Service
Chat Service
Notification Service
Moderation Service
Payment Service
Databases
Redis
Message Broker
Object Storage

91. Horizontal Scaling

The Spring Boot application should ideally remain stateless.

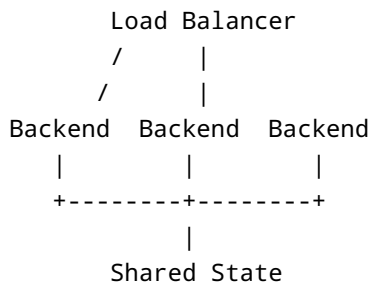
Instead of:

User session stored in JVM memory

prefer:

External session/token mechanism

Then:

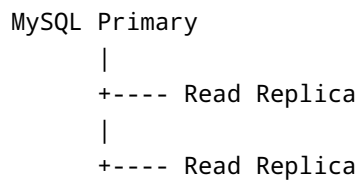


This allows additional application instances to be added.

92. Database Scaling

MySQL should initially be the source of truth.

As traffic increases:



Potentially later:

- partitioning,
- sharding,
- specialized data stores.

Do not prematurely move away from MySQL.

A well-designed MySQL architecture can support substantial early growth.

93. Data Ownership

Each module should conceptually own its data.

Example:

```
Profile Module
→ profile data

Matching Module
→ likes/matches

Chat Module
→ conversations/messages

Moderation Module
→ reports/actions
```

This makes future service extraction much easier.

94. Feature Flags

Implement feature flags from an early stage.

Example:

```
anonymous_chat_enabled
profile_reveal_enabled
new_matching_algorithm
mobile_verification_enabled
premium_enabled
ai_moderation_enabled
```

This allows features to be introduced gradually.

95. Analytics

Collect product analytics without unnecessarily collecting sensitive information.

Important events:

```
registration_started
registration_completed
verification_completed
profile_completed
anonymous_discovery_started
```

```
anonymous_connection_created  
message_sent  
reveal_requested  
reveal_accepted  
match_created  
conversation_ended  
user_reported  
user_blocked
```

Analytics should respect privacy and applicable legal requirements.

96. Privacy Architecture

The platform should follow data minimization.

Ask:

Do we really need this data?

If not, do not collect it.

Examples:

Do not require:

- exact home address,
- unnecessary contacts,
- unnecessary personal documents.

97. Data Retention

Create explicit retention policies.

Examples:

- OTP records: short-lived
- Security logs: defined retention
- Deleted accounts: defined deletion/anonymization process
- Messages: defined policy
- Reports: retained as required for safety/legal purposes

Retention periods should be finalized with appropriate legal/privacy review for the jurisdictions in which the service operates.

98. Account Deletion

Users should be able to request account deletion.

Flow:

```
Settings
  ↓
Delete Account
  ↓
Confirmation
  ↓
Account Deactivation
  ↓
Data Deletion / Anonymization
```

Some safety/audit information may need to be retained where legally required.

99. Legal/Product Documents

Before public launch, prepare:

- Terms of Service
- Privacy Policy
- Community Guidelines
- Safety Policy
- Cookie Policy where applicable
- Data deletion policy
- Reporting policy
- Age eligibility policy

Legal review should be completed before production launch.

100. Business Risks

Risk 1 – Empty Platform

If users join and find nobody available, they leave.

Solution

Launch market-by-market and build sufficient user density.

Risk 2 – Fake Accounts

Solution

- Verification
 - Rate limiting
 - Behavioral detection
 - Reports
 - Moderation
-

Risk 3 – Harassment

Solution

- Block
 - Report
 - Moderation
 - Automated risk detection
 - Strong community guidelines
-

Risk 4 – Anonymous Chat Abuse

Solution

Anonymous does not mean unaccountable.

Internally maintain:

Account Identity
+
Moderation Signals

+
Audit Trail

while keeping personal identity private from other users.

Risk 5 – Monetization Too Early

If the product immediately asks users for payment, adoption may suffer.

Solution

Build network effect first.

Risk 6 – Excessive Feature Development

Trying to compete with every dating application feature simultaneously can delay launch.

Solution

Focus on:

Anonymous conversation → mutual reveal → meaningful connection.

101. Competitive Differentiation

The product should not market itself as:

"Another Tinder."

Instead:

"A dating platform where conversation comes before identity."

Potential positioning:

Traditional Apps

Profile
↓
Photo

↓
Like
↓
Match
↓
Chat

Proposed Platform

Interest
↓
Conversation
↓
Connection
↓
Mutual Trust
↓
Profile Reveal
↓
Relationship

This is the core product story.

102. Future AI Features

AI should not be required for MVP.

Potential future features:

Conversation Suggestions

If users are struggling to start:

"Try asking about their favorite travel experience."

Compatibility Analysis

Use user-approved profile information to improve matching.

Scam Detection

Detect suspicious conversation patterns.

Moderation

Flag potentially abusive content for human review.

Profile Assistance

Help users improve:

- biography,
- interests,
- prompts.

AI should assist rather than make sensitive relationship decisions automatically.

103. Gamification

Potential future features:

- Conversation streaks
- Icebreaker challenges
- Daily questions
- Compatibility games
- Personality prompts

Avoid excessive gamification that encourages users to treat people like points or collectibles.

104. Conversation Starters

Anonymous chat can be improved using prompts.

Examples:

What is something you could talk about for hours?

What is your ideal weekend?

What's a small thing that makes your day better?

What's one place you'd love to visit?

What is a hobby you wish more people understood?

The system can rotate questions.

105. Anonymous Chat Rooms

A future extension could allow topic-based rooms:

Travel
Books
Movies
Music
Food
Career
Life
Relationships
Gaming

However, these should be introduced only after the one-to-one anonymous chat experience is stable.

106. Business Growth Phases

Phase 0 – Foundation

Build:

- architecture,
- authentication,
- database,
- security,
- CI/CD,
- frontend foundation.

Phase 1 – MVP

Build:

- profiles,
- anonymous chat,
- mutual reveal,
- matching,
- messaging,
- moderation.

Phase 2 – Growth

Build:

- better matching,
- referrals,
- notifications,
- analytics,
- improved safety,
- mobile/PWA.

Phase 3 – Scale

Build:

- Redis,
- queues,
- object storage optimization,
- horizontal scaling,
- advanced monitoring.

Phase 4 – Monetization

Build:

- subscriptions,
- premium features,
- payment system,
- billing,
- entitlement management.

Phase 5 – Microservices

Extract only the services that require independent scaling.

107. Recommended Development Roadmap

Sprint 1 – Project Foundation

- Git repository
- Spring Boot setup
- React setup
- Tailwind setup
- MySQL

- Docker
- Environment configuration
- CI pipeline
- Base exception handling
- Logging
- API documentation

Sprint 2 – Authentication

- Registration
- Login
- Logout
- Password hashing
- JWT/session mechanism
- Verification abstraction
- Mock OTP

Sprint 3 – User Profile

- Profile creation
- Profile update
- Preferences
- Privacy settings
- Photo upload foundation

Sprint 4 – Discovery

- Profile discovery
- Preference filters
- Basic ranking
- Like/pass

Sprint 5 – Anonymous Chat

- Anonymous discovery
- Connection request
- Accept/decline
- Conversation creation
- Message sending

Sprint 6 – Mutual Reveal

- Reveal request
- Accept/decline
- Profile visibility transition
- End conversation

Sprint 7 – Safety

- Block
- Report
- Moderation
- Account suspension
- Admin dashboard

Sprint 8 – Notifications

- In-app notifications
- Email architecture
- Notification preferences

Sprint 9 – Testing

- Unit tests
- Integration tests
- E2E tests
- Security testing
- Load testing

Sprint 10 – Beta

- Staging
- Monitoring
- User testing
- Bug fixes
- Performance optimization

108. Definition of MVP Success

The MVP should be considered successful when a new user can:

```
Register
↓
Verify
↓
Create Profile
↓
Find Another User
↓
Start Anonymous Conversation
↓
```

Exchange Messages
↓
Request Profile Reveal
↓
Mutually Reveal
↓
Continue Connection

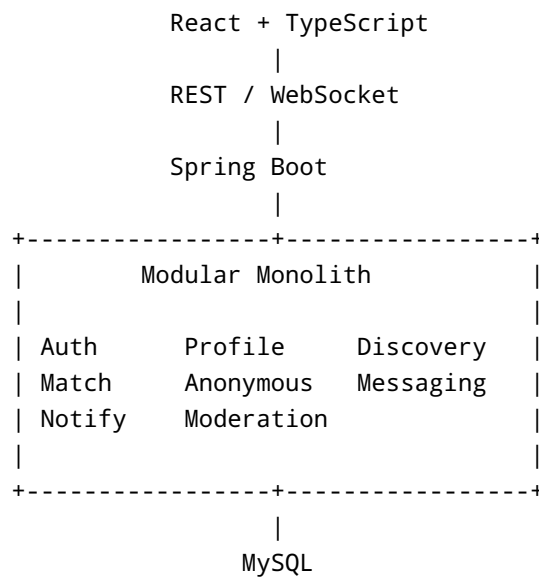
without manual intervention.

Administrators must also be able to:

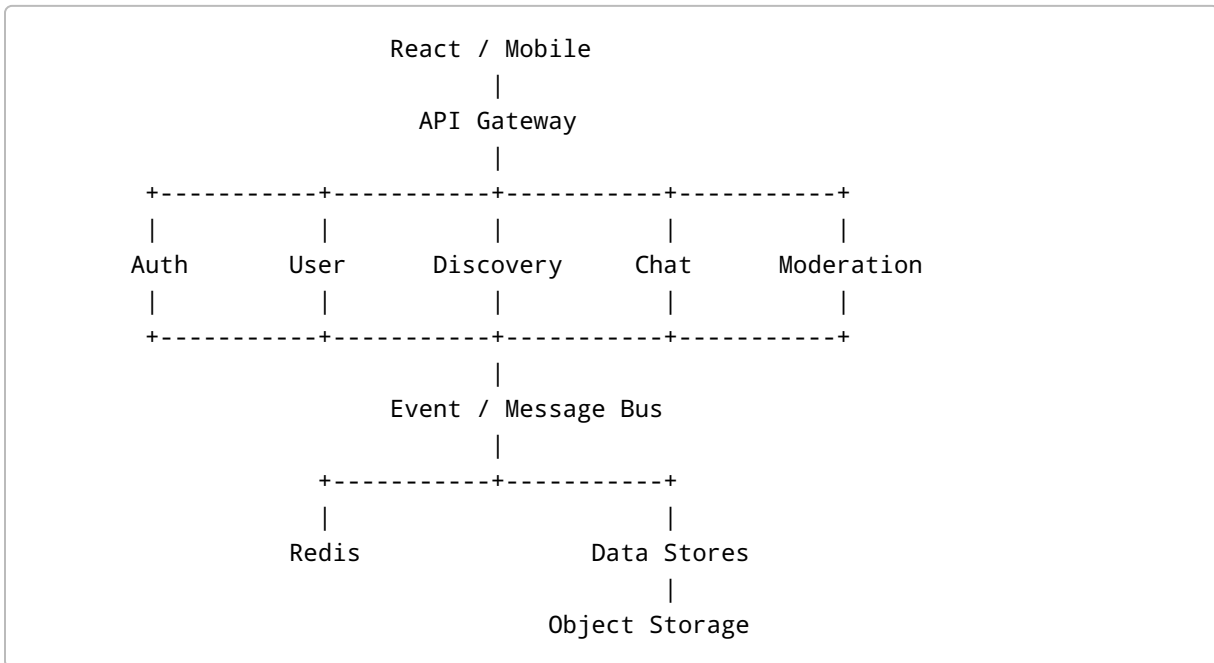
Find User
Review Report
Restrict User
Suspend User
Review Basic Audit Information

109. Product Architecture Summary

Initial



Future



110. Recommended Initial Repository

A clean repository structure could be:

```
dating-platform/
├── backend/
│   ├── src/
│   ├── pom.xml
│   └── Dockerfile
├── frontend/
│   ├── src/
│   ├── package.json
│   └── Dockerfile
├── infrastructure/
│   ├── docker/
│   ├── nginx/
│   └── scripts/
└── docs/
```

```
| | | BRD.md
| | | PRD.md
| | | API.md
| | | ARCHITECTURE.md
| | | DATABASE.md
| |
| | docker-compose.yml
| | README.md
```

111. Recommended Backend Package Structure

```
backend/src/main/java/com/example/dating/
```

```
| | | common/
| | | | | config/
| | | | | exception/
| | | | | security/
| | | | | response/
| | | | | util/
| | |
| | | auth/
| | | | | controller/
| | | | | service/
| | | | | domain/
| | | | | repository/
| | | | | dto/
| | |
| | | user/
| | | profile/
| | | discovery/
| | | matching/
| | | anonymouschat/
| | | conversation/
| | | messaging/
| | | notification/
| | | moderation/
| | | admin/
```

This structure is intentionally designed for future microservice extraction.

112. Important Architectural Rule

The most important technical decision is:

Do not build a distributed microservice system before the product has users. Build a modular monolith that behaves like a collection of well-defined services internally.

For example:

```
AnonymousChatService  
MatchingService  
ProfileService  
NotificationService
```

can initially exist inside the same Spring Boot application.

Later:

```
AnonymousChatService
```

can be extracted into its own application.

This minimizes future migration cost.

113. Product Principles for the Team

Every feature should be evaluated against these questions:

Question 1

Does this help users create meaningful connections?

Question 2

Does this respect user privacy?

Question 3

Does this require mutual consent?

Question 4

Can this feature be abused?

Question 5

Can we moderate it?

Question 6

Does it improve the free core product?

Question 7

Can the architecture scale later?

114. Final Product Definition

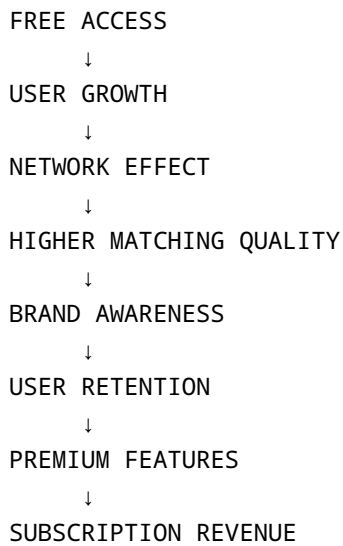
The product can ultimately be described as:

A free-first dating and social connection platform that lets adults meet through anonymous conversation before choosing whether to reveal their profiles. The platform combines anonymous conversation, traditional profile discovery and mutual matching into one privacy-first experience.

The key product loop is:

```
DISCOVER
↓
CONNECT
↓
CONVERSE
↓
BUILD TRUST
↓
MUTUAL REVEAL
↓
MATCH / CONTINUE
↓
MEANINGFUL CONNECTION
```

The business loop is:



115. Final Recommendation

The strongest version of this idea is **not to build a clone of Tinder, Bumble or Hinge**.

Instead, build a product with a very clear identity:

"Talk first. Reveal when you're ready."

The first release should concentrate heavily on three experiences:

1. **Anonymous discovery**
2. **High-quality conversation**
3. **Mutual profile revelation**

Everything else should support these three features.

Technically, the recommended path is:

```
React + TypeScript + Tailwind
      +
Spring Boot Modular Monolith
      +
MySQL
      +
WebSocket
      +
Development OTP abstraction
```

+
Strong security/moderation
+
Docker + CI/CD

Then progressively evolve:

MySQL
↓
MySQL + Redis
↓
Horizontal Spring Boot Scaling
↓
Queue/Event Architecture
↓
Selective Microservice Extraction

Do **not** change MySQL simply because the application becomes popular. First identify the actual bottleneck. MySQL can remain the primary transactional database for a substantial portion of the platform while caching, read replicas, search infrastructure, object storage and specialized stores are introduced around it.

Most importantly, **safety and privacy should be treated as core product functionality, not as features for a later release**. Anonymous communication can become the application's biggest differentiator, but it can also become its biggest risk if abuse prevention, reporting, blocking and moderation are not designed alongside it.